МИНОБРНАУКИ РОССИИ
РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М. ГУБКИНА

Факультет **Комплексной безопасности ТЭК**
Кафедра **Безопасности информационных технологий**

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

по дисциплине Языки, методы и технологии программирования

на тему Разработка паттерна для обеспечения безопасности вычислительных ресурсов суперкомпьютера при угрозе интенсивного обмена межпроцессорными сообщениями

ДАНО студенту **Мирчевски Кристина Далиборовна** (фамилия, имя, отчество в дательном падеже) группы **КС-23-18** (номер группы)

Содержание работы:

1. Введение
2. Анализ предметной области
3. Разработка паттерна
4. Разработка микросервисов
5. Тестирование и мониторинг микросервисов на кластере
6. Заключение
7. Приложение

Исходные данные для выполнения работы:

1. Информация о предметной области из открытых источников

Рекомендуемая литература:

1. Положение о курсовом проектировании в РГУ нефти и газа (НИУ) имени И.М. Губкина (приказ от 11.11.2021 № 398)
2. Docker Compose Documentation [Электронный ресурс]: сайт. – Режим доступа: https://docs.docker.com/compose/
3. Kubernetes Documentation [Электронный ресурс]: сайт. – Режим доступа: https://kubernetes.io/docs/home/

Графическая часть:

1. Модель угрозы; 2. Механизм защиты; 3. Архитектура паттерна; 4. UML диаграммы; 5. Коды микросервисов; 6. Витрины с результатами мониторинга и тестирования микросервисов на кластере

Требования к представлению результатов:

Электронная версия

Руководитель: д.т.н. (уч.степень) профессор (должность) (подпись) Корнеев Николай Владимирович (фамилия, имя, отчество)

Задание принял к исполнению: студент (подпись) Мирчевски Кристина Далиборовна (фамилия, имя, отчество)

МИНОБРНАУКИ РОССИИ

РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М. ГУБКИНА

Факультет **Комплексной безопасности ТЭК**
Кафедра **Безопасности информационных технологий**

Оценка комиссии: _____ Рейтинг: ____
Подписи членов комиссии:

_____ _____
(подпись) (фамилия, имя, отчество)

_____ _____
(подпись) (фамилия, имя, отчество)

_____
(дата)

КУРСОВАЯ РАБОТА

по дисциплине Языки, методы и технологии программирования

на тему Разработка паттерна для обеспечения безопасности вычислительных ресурсов суперкомпьютера при угрозе интенсивного обмена межпроцессорными сообщениями

«К ЗАЩИТЕ»

Профессор, д.т.н. Корнеев Н.В.
(должность, ученая степень; фамилия, и.о.)

_____
(подпись)

_____
(дата)

ВЫПОЛНИЛ:
Студент группы **КС-23-18**
(номер группы)

Мирчевски Кристина Далиборовна
(фамилия, имя, отчество)

_____
(подпись)

_____
(дата)

Москва, 20 25

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 3
2. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ..... 6
2.1 Модель угрозы..... 7
2.2 Механизм защиты..... 11
3. РАЗРАБОТКА ПАТТЕРНА..... 15
3.1 Архитектура паттерна..... 15
3.2 UML-диаграммы..... 18
4. РАЗРАБОТКА МИКРОСЕРВИСОВ..... 22
4.1 Функции микросервисов..... 22
4.2 Разработка микросервисов..... 28
5. МОНИТОРИНГ МИКРОСЕРВИСОВ..... 32
5.1 Тестирование микросервисов..... 32
5.2 Мониторинг микросервисов..... 34
ЗАКЛЮЧЕНИЕ..... 36
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ..... 38

ВВЕДЕНИЕ

Современный этап научно-технического прогресса характеризуется возрастающей зависимостью от высокопроизводительных вычислений. Суперкомпьютеры, представляющие собой специализированные мощные вычислительные машины, значительно превосходящие по своим техническим параметрам и скорости вычислений компьютеры общего пользования, и объединяющие тысячи процессорных ядер и специализированные сети межсоединений, стали ключевым инструментом для решения фундаментальных и прикладных задач в разных областях, таких как биология, медицина, космическое пространство, промышленность, машинное обучение и других. Эффективность этих систем определяется не только пиковой производительностью их компонентов, но и сбалансированностью архитектуры, где критически важную роль играет сеть межпроцессорных коммуникаций.

Однако стремительный рост производительности этих вычислительных гигантов, достигаемый за счет увеличения количества вычислительных узлов и усложнения их коммуникационной инфраструктуры, порождает принципиально новые вызовы в области информационной безопасности. Парадигма безопасности традиционных информационных систем оказывается недостаточно эффективной в среде, где ключевым процессом является не обработка транзакций, а интенсивный, лавинообразный обмен данными между тысячами параллельно работающих процессов. В этом контексте особую актуальность приобретает угроза нарушения безопасности, исходящая не извне, а изнутри системы. Её реализация может привести к лавинообразной перегрузке сети межсоединений, что ставит под удар стабильность работы всего вычислительного комплекса. Это влечет за собой серьезные финансовые потери, срывы научных исследований и коммерческих проектов, что и определяет высокую актуальность разработки специализированных механизмов защиты.

Целью данной курсовой работы является разработка паттерна (шаблона) для обеспечения доступности вычислительных ресурсов суперкомпьютера, обеспечивающего защиту от угроз, связанных с интенсивным обменом межпроцессорными сообщениями.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ современных угроз безопасности суперкомпьютерных систем, связанных с интенсивным обменом МПС.
2. Смоделировать угрозы нарушения на основе методик ФСТЭК, определив источники, уязвимости и сценарий реализации.
3. Спроектировать паттерн безопасности, определив его архитектуру, компоненты и алгоритмы взаимодействия.
4. Предложить конкретный механизм защиты и описать схему его работы.
5. Оценить практическую ценность и эффективность предложенного решения.

Объектом исследования является процесс обеспечения информационной безопасности вычислительных ресурсов суперкомпьютерных систем в условиях интенсивного обмена межпроцессорными сообщениями.

Предметом исследования выступают паттерны и механизмы защиты вычислительных ресурсов суперкомпьютера от угроз, связанных с аномально интенсивным обменом межпроцессорными сообщениями.

Новизна работы заключается в создании комплексного решения для защиты суперкомпьютера от перегрузки сетевых соединений между процессорами. В отличие от существующих подходов предлагаемый паттерн сфокусирован исключительно на угрозах в подсистеме межпроцессорных коммуникаций, что отличает его от общих решений для сетевой безопасности. По сути, создается интеллектуальный механизм защиты, который не просто собирает данные о сетевой нагрузке, а самостоятельно принимает решения и останавливает задачи-нарушители до того, как они

выведут из строя всю вычислительную систему. Такой проактивный подход к обеспечению корректной работы вычислительных ресурсов представляет собой качественно новый уровень безопасности суперкомпьютерных инфраструктур.

Практическая ценность работы состоит в том, что ее результаты могут быть непосредственно внедрены в эксплуатацию суперкомпьютерных центров. Разработанный паттерн позволит:

- повысить отказоустойчивость и общую доступность вычислительных ресурсов.
- снизить время простоя кластера и минимизировать финансовые и репутационные издержки.
- автоматизировать процесс реагирования на инциденты, связанные с перегрузкой сети, снизив нагрузку на системных администраторов.
- создать основу для построения более сложных систем безопасности, использующих машинное обучение для прогнозирования аномалий.

Курсовая работа состоит из трёх разделов, заключения, списка использованных источников и приложений.

2. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Формирование эффективного паттерна защиты вычислительных ресурсов суперкомпьютера требует первоначального выявления спектра угроз, возникающих при интенсивном обмене межпроцессорными сообщениями (далее — МПС). Этот этап является фундаментальным, поскольку именно от полноты идентификации угроз зависит адекватность и эффективность разрабатываемого механизма безопасности.

В соответствии с методикой ФСТЭК России, отправной точкой выступает построение модели угроз, которая должна отражать ключевые компоненты: объекты воздействия, источники угроз, сценарии реализации атак через систему МПС, а также потенциальный ущерб для вычислительного процесса.

Предметная область исследования охватывает вопросы обеспечения информационной безопасности высокопроизводительных вычислительных систем в условиях экстремальных нагрузок на подсистему межпроцессорных коммуникаций. Вычислительные узлы, высокоскоростная коммуникационная сеть, планировщики заданий и системное программное обеспечение образуют комплекс взаимосвязанных компонентов, уязвимых к атакам на доступность и целостность вычислений.

Особое внимание уделяется защите каналов передачи МПС, поскольку именно они становятся ключевым вектором атак, способным парализовать работу всего суперкомпьютерного комплекса. Интенсивный, аномальный трафик МПС может привести к нарушению базовых свойств безопасности - доступности вычислительных ресурсов и целостности результатов обработки данных.

Рассмотрим детализированную модель угроз безопасности информации применительно к исследуемой предметной области, сфокусировавшись на двух основных сценариях: нарушении доступности и нарушении целостности вычислительного процесса.

2.1 Модель угрозы

Сценарий 1. Нарушение доступности вычислительных ресурсов суперкомпьютера через насыщение коммуникационной сети.

Название угрозы: нарушение доступности вычислительных ресурсов через генерацию интенсивного трафика межпроцессорных сообщений (МПС-флуд).

Моделирование угрозы нарушения доступности выполнено в соответствии с «Методикой оценки угроз безопасности информации» [1]. Сценарий описывает атаку типа «отказ в обслуживании» (Denial of Service), направленную на нарушение работоспособности суперкомпьютерного комплекса.

Источником угрозы выступает внутренний нарушитель с легитимным доступом (пользователь системы), использующий выделенные ему вычислительные узлы для запуска вредоносного задания. Объектами воздействия являются ключевые компоненты суперкомпьютерной инфраструктуры: высокоскоростная коммуникационная сеть, коммутаторы, сетевые адаптеры вычислительных узлов, а также планировщик заданий.

Реализация угрозы начинается с инициации специализированного вычислительного задания, в котором процессы на различных узлах начинают интенсивный обмен служебными или бессмысленными сообщениями. На первом этапе осуществляется насыщение пропускной способности каналов связи между узлами, что приводит к увеличению задержек и потери пакетов. Одновременно происходит переполнение буферов сетевых адаптеров и коммутаторов, вызывая исчерпание их ресурсов обработки. Возросшая сетевая активность негативно влияет на работу планировщика заданий, который фиксирует аномальное поведение системы, но не может корректно распределить ресурсы между конкурирующими задачами.

На рисунке представлена схема реализации угрозы нарушения доступности при МПС-атаке на суперкомпьютерную инфраструктуру.

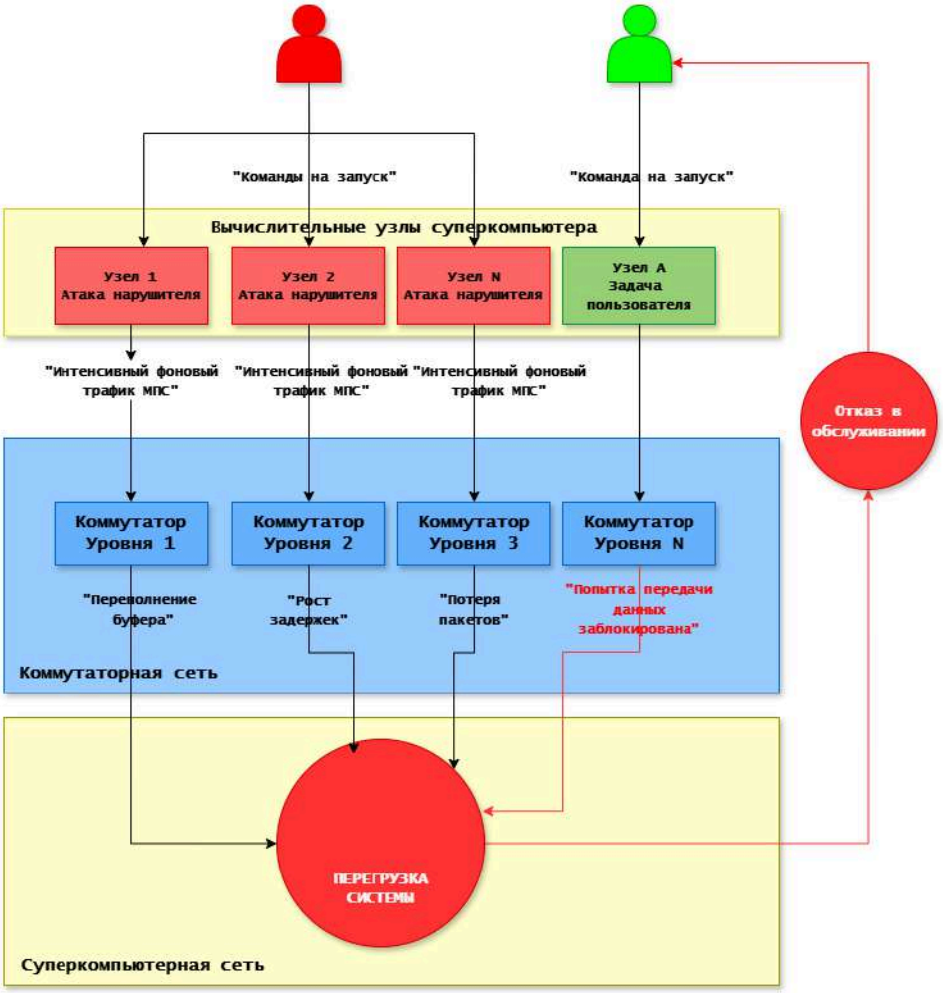


Рисунок 1 – Сценарий нарушения доступности вычислительных ресурсов суперкомпьютера через насыщение коммуникационной сети

Критическим последствием атаки становится полная или частичная недоступность вычислительных мощностей для легитимных задач. Пользователи при попытке выполнения расчетов сталкиваются с значительным ростом времени выполнения задач или их полным зависанием из-за невозможности организации межпроцессорного взаимодействия. Результатом успешной реализации угрозы является нарушение доступности вычислительных сервисов, приводящее к срыву научных экспериментов, значительным финансовым потерям от простоя дорогостоящего оборудования и репутационному ущербу для вычислительного центра. Данный сценарий демонстрирует уязвимость суперкомпьютерных систем к целенаправленным атакам на подсистему коммуникаций и необходимость

реализации специализированных механизмов контроля и управления трафиком МПС.

Сценарий 2. Нарушение целостности вычислений через навязывание некорректных межпроцессорных сообщений.

Название угрозы: нарушение целостности результатов вычислений через компрометацию узла и модификацию передаваемых данных.

Моделирование угрозы нарушения целостности выполнено в соответствии с «Методикой оценки угроз безопасности информации» [1]. Сценарий описывает целевое воздействие на вычислительный процесс, направленное на искажение результатов параллельных расчетов.

Источником угрозы выступает внутренний нарушитель с привилегированным доступом (злоумышленник, получивший контроль над узлом через уязвимость в системном ПО) или внешний нарушитель, использующий каналы удаленного управления. Объектами воздействия являются вычислительные узлы, выполняющие части параллельной задачи, каналы передачи данных между процессами, а также итоговые результаты вычислений.

Реализация угрозы начинается с компрометации одного или нескольких вычислительных узлов, на которых выполняются процессы легитимной задачи. На первом этапе осуществляется перехват управляющих или вычислительных данных, передаваемых между процессами. На втором этапе производится модификация содержимого сообщений — изменение численных значений в матрицах, искажение управляющих параметров алгоритма или подмена блоков данных. На третьем этапе искаженная информация передается соседним процессам, которые продолжают вычисления на основе некорректных данных. Особую опасность представляют атаки на системы синхронизации, которые могут вызывать каскадные сбои во всех параллельных вычислениях.

На рисунке представлена схема реализации угрозы нарушения целостности через навязывание некорректных данных в суперкомпьютерной системе.



Рисунок 2 – Нарушение целостности вычислений через навязывание некорректных межпроцессорных сообщений

Критическим последствием атаки становится получение научно или технически неверных результатов при формально успешном завершении расчетов. Пользователи могут длительное время не обнаруживать факт компрометации, используя искаженные данные для принятия ответственных решений в таких областях, как проектирование сложных инженерных конструкций, фармакология или климатическое моделирование. Результатом успешной реализации угрозы является нарушение целостности вычислений, приводящее к принятию ошибочных научных выводов, созданию небезопасных технических решений и полной дискредитации результатов работы вычислительного центра. Данный сценарий демонстрирует уязвимость суперкомпьютерных систем к целевым атакам на достоверность

вычислений и необходимость реализации механизмов верификации передаваемых данных и контроля целостности вычислительного процесса.

2.2 Механизм защиты

Сценарий 1. Нарушение доступности вычислительных ресурсов суперкомпьютера через насыщение коммуникационной сети.

Разработанный механизм защиты от атак типа «отказ в обслуживании» реализует многоуровневую систему мониторинга и управления трафиком МПС. Основу защиты составляет сервис безопасности, осуществляющий непрерывный мониторинг и анализ всего трафика МПС в реальном времени.

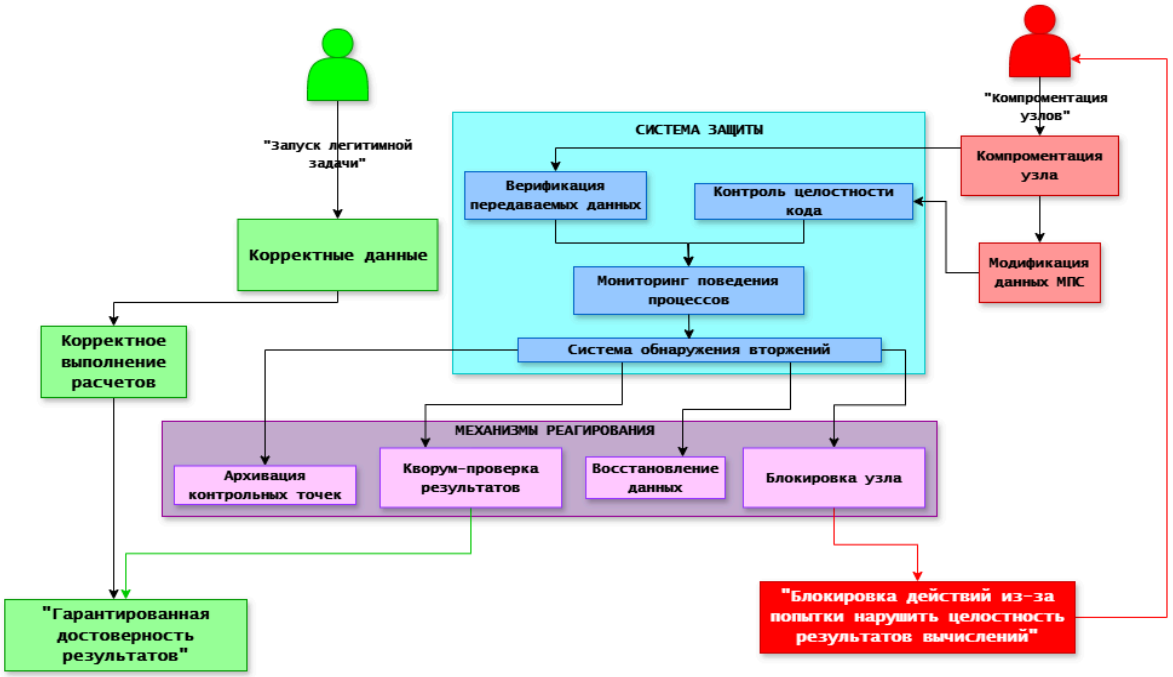


Рисунок 3 – Схема механизма защиты от нарушения доступности вычислительных ресурсов суперкомпьютера через насыщение коммуникационной сети

При поступлении аномального трафика МПС от задания, инициированного внутренним нарушителем, сервис безопасности проводит глубокий анализ паттернов обмена сообщениями на предмет соответствия эталонным профилям легитимных вычислительных заданий. Для идентификации атак на доступность применяются алгоритмы обнаружения аномалий, включая: статистический анализ частоты и объема

межпроцессорных сообщений, мониторинг использования пропускной способности коммуникационной сети, выявление признаков целенаправленного создания сетевых коллизий (аномалий), анализ поведения планировщика заданий и менеджера ресурсов.

Механизм обеспечивает автоматическое разделение трафика на два потока: выявленный легитимный трафик МПС направляется через приоритетные каналы коммуникационной сети, в то время как вредоносные сообщения подвергаются немедленному ограничению.

Процедура блокировки включает: изоляцию вредоносных процессов, ограничение трафика, применяя лимиты на частоту обмена сообщениями, автоматическую приостановку заданий, нарушающих политики безопасности, и уведомление администратора о попытке нарушения доступности.

Критическим компонентом системы является интеграция с планировщиком заданий, который обеспечивает равномерное распределение вычислительных ресурсов между легитимными пользователями, предотвращая перегрузку коммуникационной инфраструктуры даже в условиях целенаправленной атаки.

Защищенные вычислительные задания функционируют в изолированных сегментах коммуникационной сети с гарантированным выделением ресурсов, что обеспечивает непрерывность выполнения научных расчетов легитимными пользователями.

Данный механизм демонстрирует высокую эффективность против атак на доступность вычислительных ресурсов, обеспечивая непрерывность научных исследований за счет комбинации превентивного мониторинга, интеллектуального анализа трафика МПС и автоматизированного реагирования на угрозы в реальном времени.

Ключевые преимущества: минимизация ложных срабатываний при идентификации легитимных заданий, сохранение производительности системы при отражении атак, интеграция с существующей инфраструктурой

суперкомпьютера, масштабируемость для систем с тысячами вычислительных узлов

Сценарий 2. Нарушение целостности вычислений через навязывание некорректных данных.

Разработанный механизм защиты от атак на целостность вычислений реализует многоуровневую систему верификации и контроля достоверности межпроцессорных сообщений. Основу защиты составляет сервис мониторинга целостности данных, осуществляющий непрерывный контроль корректности передаваемых данных между вычислительными узлами.

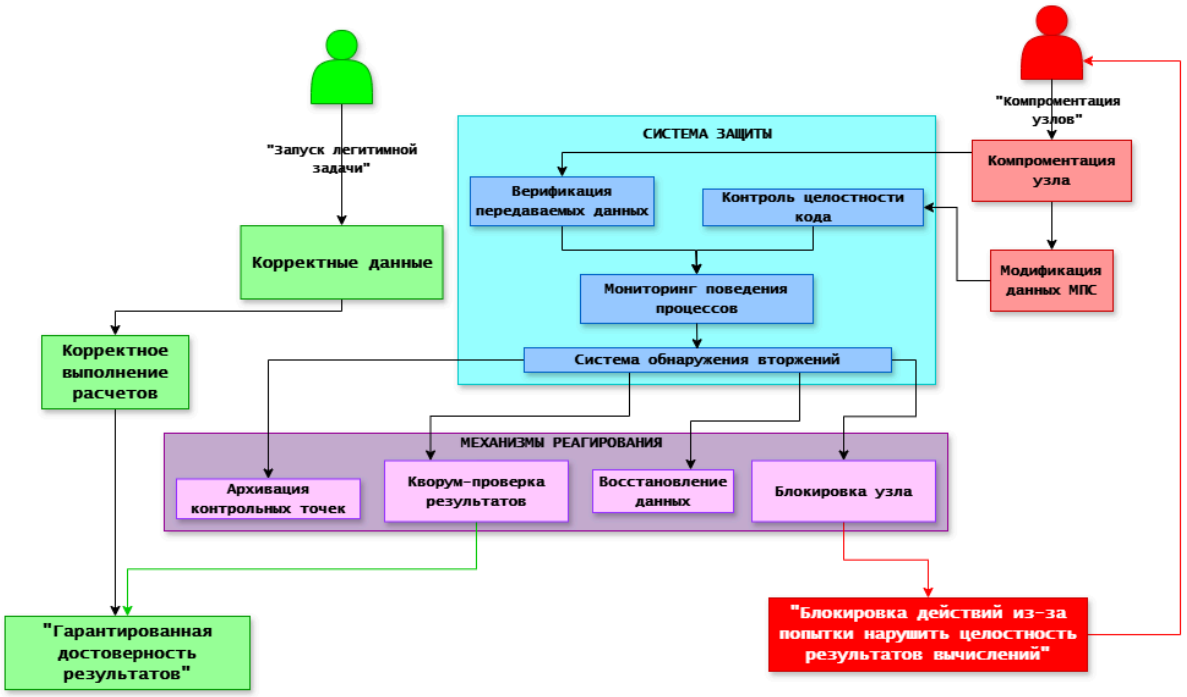


Рисунок 4 – Схема механизма защиты от нарушения целостности вычислений через навязывание некорректных межпроцессорных сообщений

Принципы работы механизма защиты: при попытке компрометации вычислительного узла и модификации передаваемых данных сервис безопасности проводит комплексную проверку целостности информации на всех этапах вычислительного процесса. Для идентификации атак на целостность применяются следующие механизмы: контроль целостности кода, мониторинг согласованности данных между реплицирующимися процессами, анализ корректности передаваемых параметров.

Механизм обеспечивает многоуровневую верификацию данных: на уровне отдельного сообщения, на уровне блока данных и на уровне всего вычислительного процесса. Выявленные корректные данные продолжают участие в вычислениях, в то время как модифицированные сообщения немедленно отбрасываются.

Процедура защиты целостности включает: регулярное создание контрольных точек вычислений, восстановление из последней корректной точки, сравнение результатов параллельных вычислений в разных доменах, блокировку узлов с признаками компрометации.

Критическим компонентом системы является архитектура кворум-проверки, которая обеспечивает независимую верификацию результатов вычислений несколькими изолированными вычислительными доменами, предотвращая влияние скомпрометированных узлов на финальный результат.

Защищенные вычислительные процессы выполняются в условиях гарантированной целостности данных, что обеспечивает достоверность научных результатов и корректность инженерных расчетов для легитимных пользователей.

Данный механизм демонстрирует высокую эффективность против целевых атак на целостность вычислений, обеспечивая надежность научных исследований за счет комбинации превентивного контроля, многоуровневой верификации и автоматизированного восстановления после инцидентов.

Ключевые преимущества: минимизация рисков получения некорректных научных результатов, автоматическое восстановление после попыток компрометации, сохранение производительности за счет оптимизированных алгоритмов проверки, масштабируемость для распределенных вычислительных систем.

3. РАЗРАБОТКА ПАТТЕРНА

3.1 Архитектура паттерна

Разработанный паттерн безопасности представляет собой микросервисную архитектуру, ориентированную на обеспечение доступности и целостности вычислительных ресурсов суперкомпьютера в условиях угроз, связанных с интенсивным обменом МПС. Архитектура построена на принципах модульности, слабой связанности и высокой степени автоматизации, что позволяет эффективно распределить функции защиты между специализированными сервисами.

Выбор микросервисной архитектуры обусловлен следующими факторами:

— Масштабируемость: каждый сервис может масштабироваться независимо в зависимости от нагрузки, что критически важно для суперкомпьютерных систем с тысячами узлов.

— Устойчивость к сбоям: отказ одного сервиса не приводит к полному коллапсу системы защиты.

— Гибкость разработки и развертывания: разные команды могут параллельно разрабатывать и совершенствовать отдельные сервисы, следуя гибким методологиям (SCRUM). Каждая команда четко знает контракт своего сервиса: какие данные поступают на вход и какой результат ожидается на выходе.

— Технологическая гибкость: для каждого сервиса может быть выбрана наиболее подходящая технологическая стека (например, Python для ML-анализа, Go для высоконагруженного мониторинга трафика).

Архитектура паттерна включает четыре ключевых микросервисов:

1. Traffic Monitor Service (Сервис мониторинга трафика):

Назначение сервиса заключается в постоянном сборе и агрегации метрик сетевого трафика межпроцессорных сообщений (МПС) в реальном

времени. Сервис работает в пассивном режиме, осуществляя мониторинг сетевой активности на вычислительных узлах.

На вход поступают низкоуровневые данные о сетевых пакетах и потоках МПС, собираемые с сетевых интерфейсов узлов.

Выходными данными являются обработанные и структурированные метрики производительности сети, которые передаются для дальнейшего анализа в систему безопасности.

2. Security Analyzer Service (Сервис анализа безопасности):

Основная функция этого сервиса — интеллектуальный анализ поступающих метрик и выявление потенциальных угроз безопасности. Сервис использует комбинацию алгоритмов машинного обучения и сигнатурного анализа для обнаружения аномальных паттернов в трафике МПС.

На вход сервис получает агрегированные метрики от Traffic Monitor Service.

На выходе формируются данные об обнаруженных угрозах, включая их классификацию, оценку критичности и рекомендации по реагированию, которые передаются другим компонентам системы защиты.

3. Policy Manager Service (Сервис управления политиками):

Данный сервис отвечает за централизованное управление правилами и политиками безопасности во всей системе. Его назначение включает хранение, валидацию и распространение политик безопасности, определяющих условия срабатывания защитных механизмов и соответствующие контрмеры.

На вход сервис получает запросы на проверку и применение политик, а также команды на их обновление.

Выходными результатами являются актуальные правила безопасности, применяемые для оценки инцидентов и определения адекватных ответных действий системой защиты.

4. Audit & Logging Service (Сервис аудита и логирования):

Назначение сервиса заключается в обеспечении комплексного учёта и документирования всех событий безопасности в системе. Сервис осуществляет централизованный сбор, обработку и долгосрочное хранение логов от всех компонентов системы защиты.

На вход поступают структурированные записи о событиях безопасности, действиях пользователей и системных операциях.

Выходными данными являются агрегированные журналы событий, готовые для анализа, формирования отчётов и проведения аудита безопасности, что обеспечивает подотчётность и возможность расследования инцидентов.



Рисунок 5 – Микросервисная архитектура паттерна обеспечения доступности и целостности вычислительных ресурсов суперкомпьютера в условиях угроз, связанных с интенсивным обменом МПС

Схема работы паттерна:

1. Traffic Monitor Service (Сервис мониторинга трафика) непрерывно собирает метрики сетевого трафика МПС с вычислительных узлов суперкомпьютера, агрегирует данные и передает их Security Analyzer Service (Сервису анализа безопасности) для дальнейшей обработки. Все операции сбора данных фиксируются в системе логирования.
2. Security Analyzer Service (Сервис анализа безопасности) анализирует полученные метрики с использованием алгоритмов машинного обучения и сигнатурного анализа. Если аномальная активность не

обнаружена, процесс возвращается к шагу 1 для продолжения мониторинга. При выявлении потенциальной угрозы безопасности сервис инициирует инцидент, определяя его тип, уровень критичности и идентификаторы затронутых ресурсов. Информация об обнаруженной угрозе заносится в журналы безопасности.

3. Policy Manager Service (Сервис управления политиками), получив данные об инциденте, определяет применимые политики безопасности и соответствующие контрмеры. На основе утвержденных политик сервис генерирует команды управления, которые передаются внешним системам суперкомпьютера: планировщику заданий (для приостановки или остановки вредоносных заданий) и сетевым коммутаторам (для изоляции скомпрометированных узлов или применения механизмов контроля качества обслуживания). Все действия по реагированию документируются в системе аудита.
4. Audit & Logging Service (Сервис аудита и логирования) обеспечивает фиксацию всего жизненного цикла инцидента — от момента обнаружения до выполнения контрмер. Сервис собирает структурированные логи от всех компонентов системы, обеспечивая целостность и доступность данных для последующего анализа, формирования отчетов и проведения аудита безопасности. Администратор системы имеет возможность вмешаться в процесс автоматического реагирования при необходимости корректировки действий или проведения ручного расследования инцидента.

Все микросервисы развертываются в контейнеризированной среде (Docker), что обеспечивает их переносимость и простоту управления в гетерогенной среде суперкомпьютерного центра.

3.2 UML-диаграммы

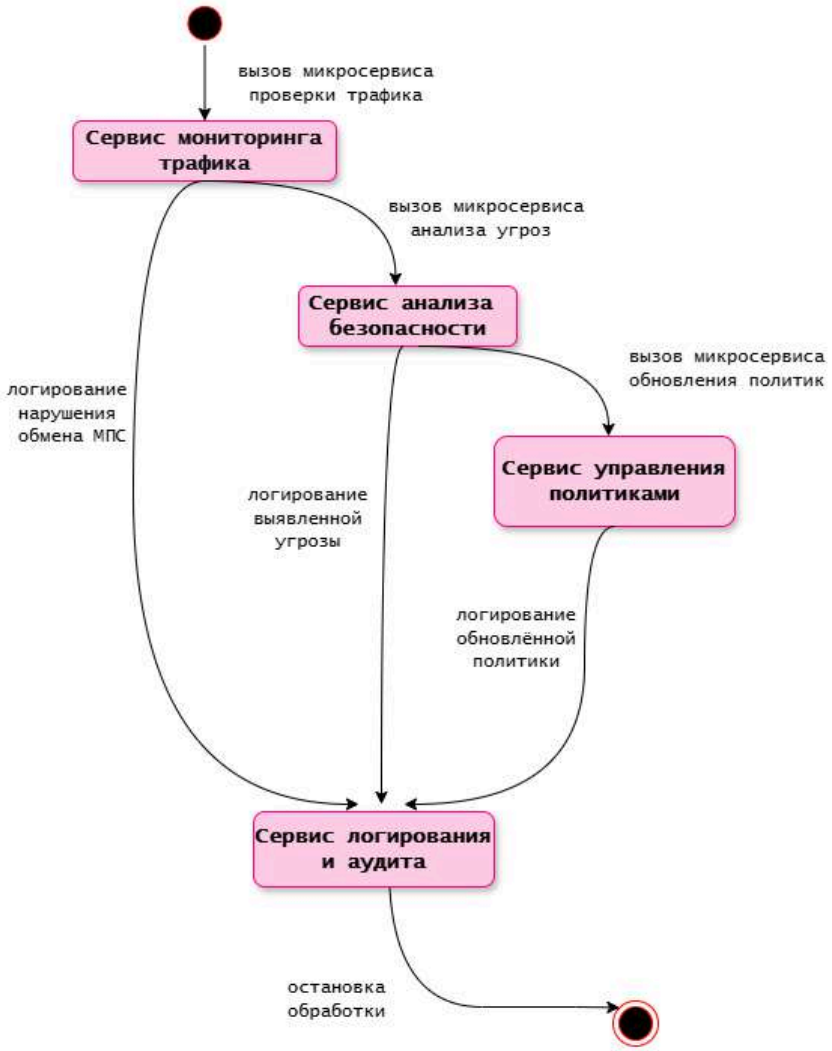


Рисунок 6 – UML-диаграмма состояния паттерна безопасности

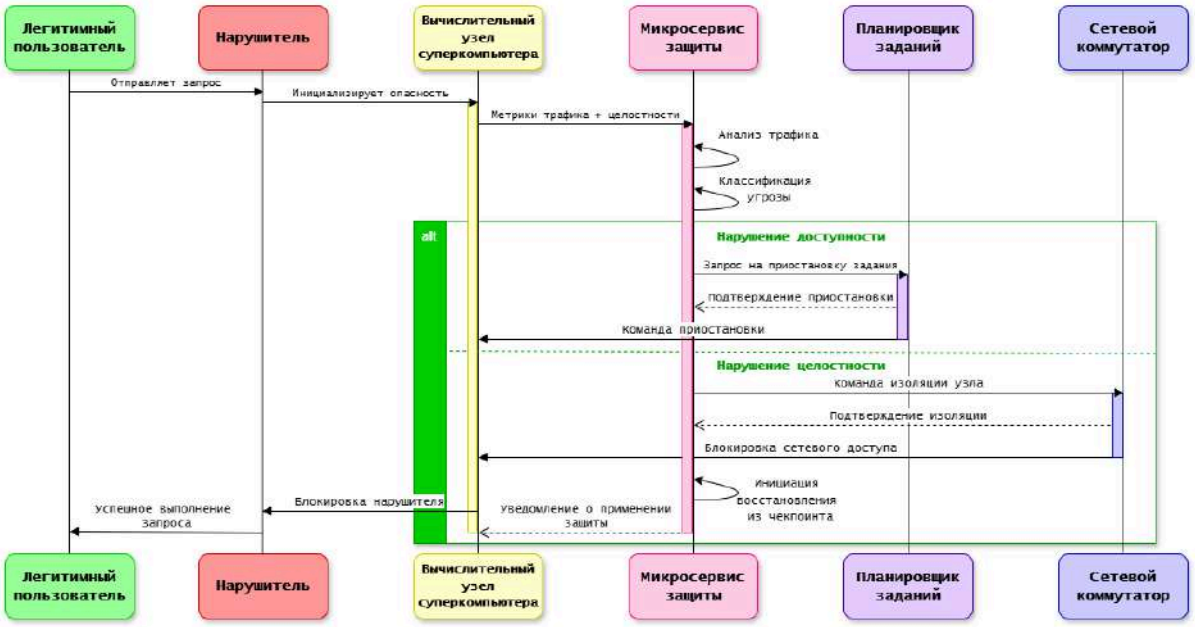


Рисунок 7 – UML-диаграмма последовательности паттерна безопасности

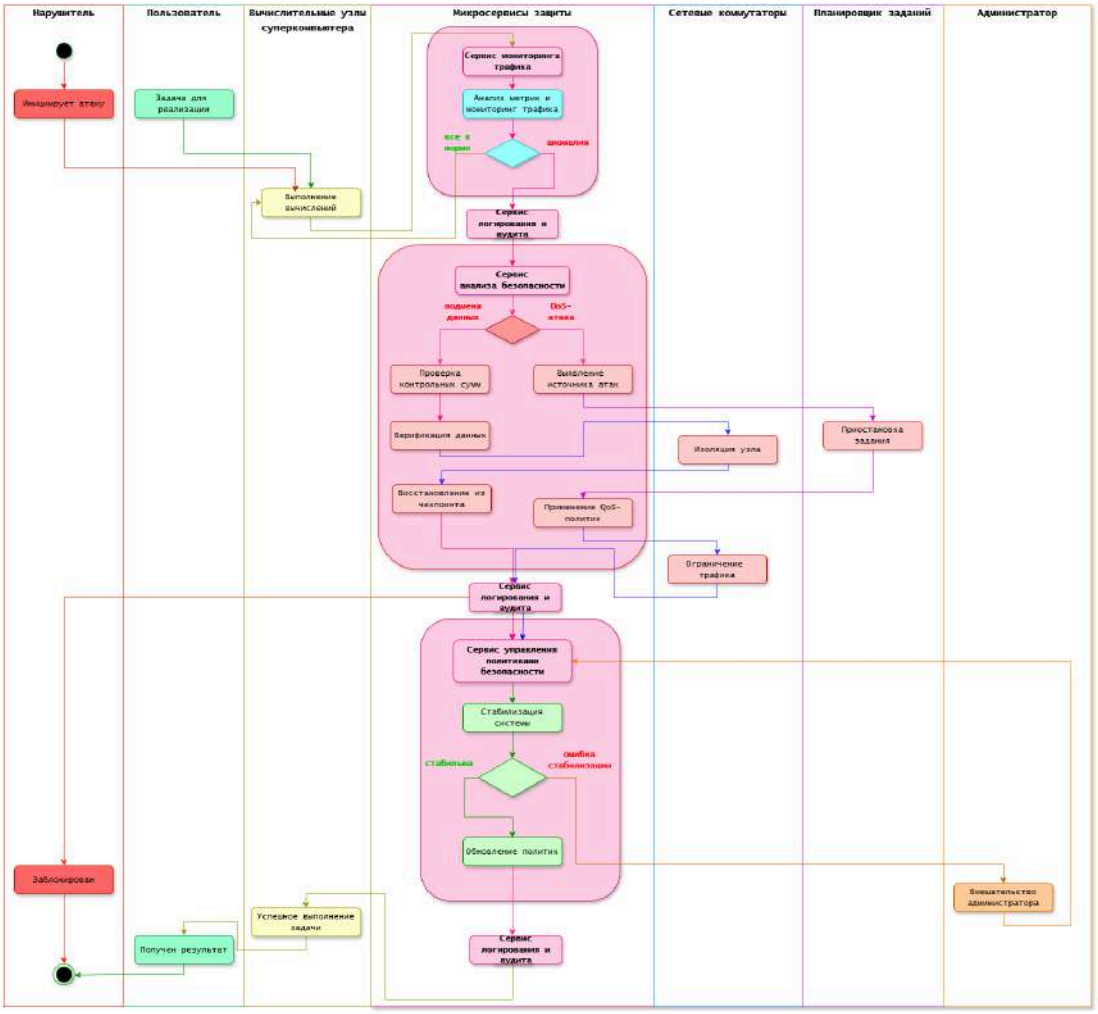


Рисунок 8 – UML-диаграмма деятельности паттерна безопасности

4. РАЗРАБОТКА МИКРОСЕРВИСОВ

4.1 Функции микросервисов

В рамках разработанного паттерна безопасности для защиты вычислительных ресурсов суперкомпьютера реализована микросервисная архитектура, состоящая из четырех специализированных сервисов, взаимодействующих через шину сообщений Apache Kafka. Каждый микросервис выполняет строго определенный набор функций и развертывается в контейнеризированной среде Docker с оркестрацией через Kubernetes, что обеспечивает высокую степень отказоустойчивости, масштабируемости и простоты управления в гетерогенной инфраструктуре суперкомпьютерного центра. Микросервисы разработаны на языке программирования Python.

Traffic Monitor Service (Сервис мониторинга трафика) выполняет критически важную функцию пассивного сбора и агрегации низкоуровневых метрик трафика межпроцессорных сообщений (МПС) в реальном времени. Сервис развертывается как DaemonSet на каждом вычислительном узле кластера и осуществляет мониторинг сетевых интерфейсов InfiniBand (ib0) с автоматическим переходом на резервные Ethernet-интерфейсы при необходимости. Основные операции включают периодический опрос сетевых статистик, вычисление ключевых показателей производительности (пакетов в секунду, байт в секунду), буферизацию данных и отправку агрегированных метрик в Kafka-топик mpi-metrics. Сервис реализует асинхронную модель работы с использованием библиотеки aiokafka для минимизации нагрузки на вычислительные ресурсы и включает механизмы обработки ошибок с экспоненциальным откатом при временной недоступности Kafka.

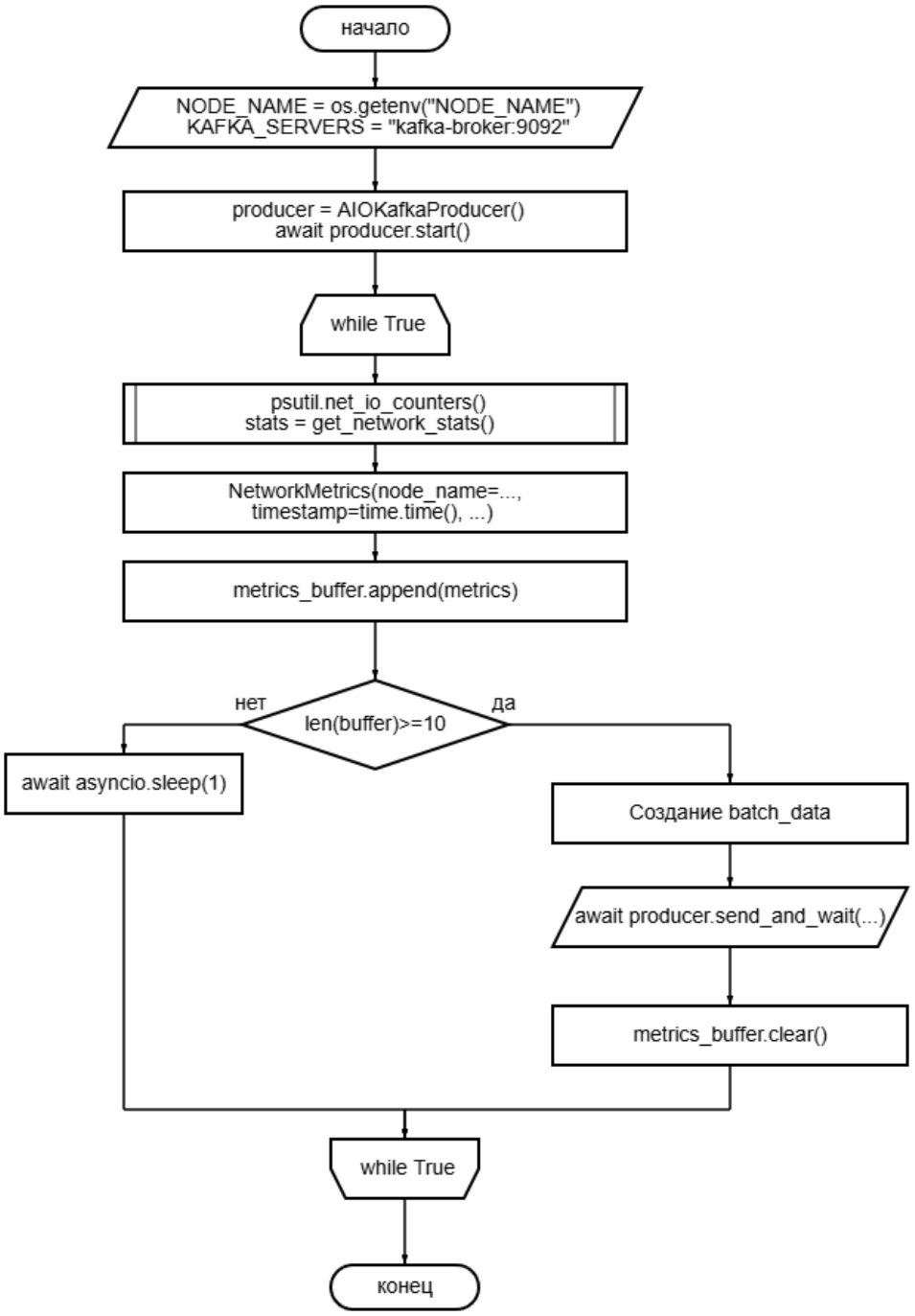


Рисунок 9 – Блок-схема сервиса мониторинга трафика на Python

Security Analyzer Service (Сервис анализа безопасности) представляет собой интеллектуальный ядро системы защиты, объединяющее функции обнаружения угроз и автоматического реагирования. Сервис потребляет поток метрик из Kafka и применяет алгоритмы машинного обучения, включая модель Isolation Forest, для выявления аномальных паттернов трафика, характерных для атак типа MPI-флуд. При обнаружении угрозы сервис

создает структурированный инцидент, определяет применимую политику безопасности через Policy Manager и автоматически выполняет контрмеры: приостановку вредоносных заданий через интеграцию с планировщиком Slurm или изоляцию скомпрометированных узлов через SDN-контроллер. Реализация включает продвинутую retry-логику с экспоненциальным backoff для устойчивости к временным сбоям внешних систем.

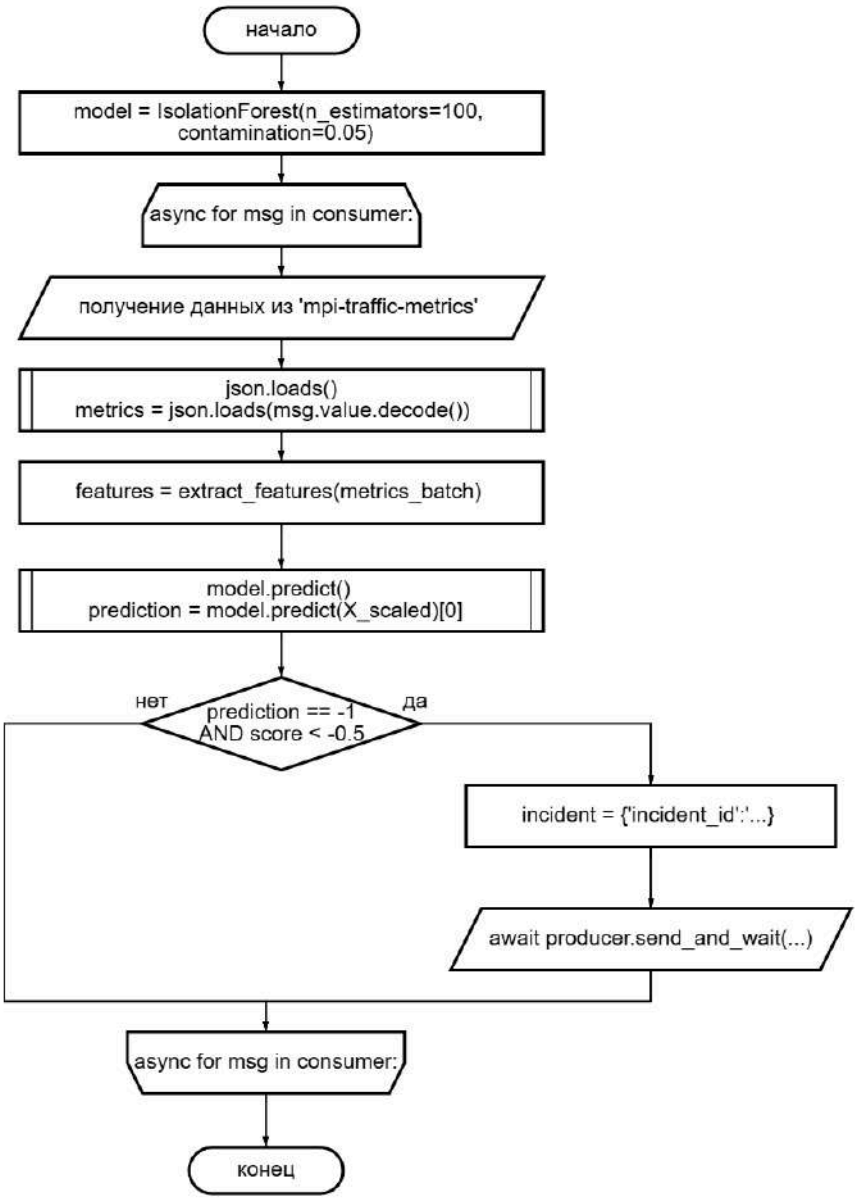


Рисунок 10 – Блок-схема сервиса анализа безопасности на Python

Policy Manager Service (Сервис управления политиками) отвечает за централизованное хранение, управление и распространение правил безопасности в системе. Сервис загружает политики из YAML-конфигураций

или базы данных, обеспечивает их валидацию на корректность структуры и логическую целостность, предоставляет REST API для динамического обновления правил администраторами. Политики определяют условия срабатывания, типы контрмер (приостановка задания, изоляция узла, ограничение трафика) и параметры severity для различных категорий угроз. Сервис интегрируется с Security Analyzer для оперативного предоставления актуальных правил реагирования и поддерживает механизмы версионирования для отслеживания изменений в политиках безопасности.

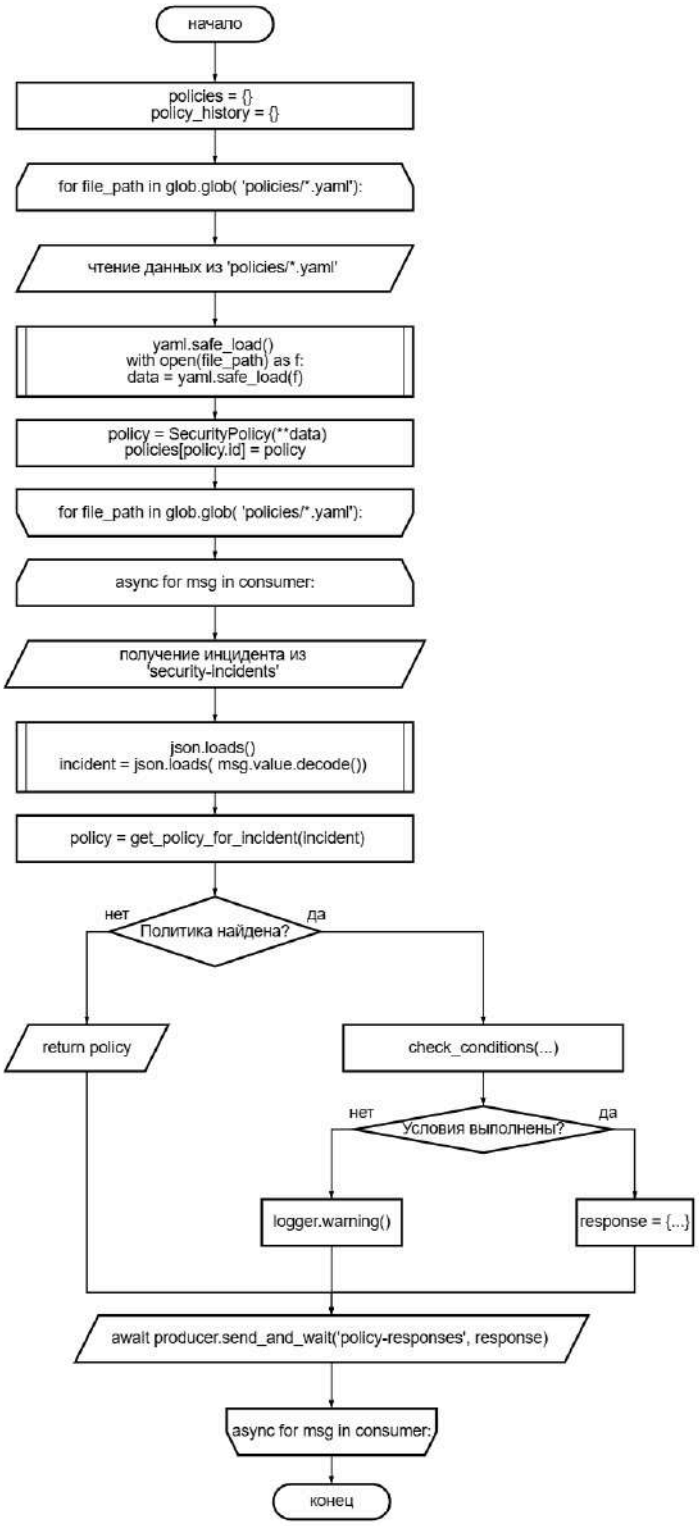


Рисунок 11 – Блок-схема сервиса управления политиками на Python

Audit & Logging Service (Сервис аудита и логирования) обеспечивает централизованный сбор, обработку, хранение и предоставление доступа ко всем событиям системы безопасности. Сервис принимает структурированные логи от всех компонентов через Kafka, обогащает их метаданными

(временными метками, идентификаторами служб) и сохраняет в Elasticsearch с автоматическим созданием индексов по датам для упрощения ротации и анализа. Параллельно сервис реализует REST API для веб-интерфейса мониторинга, предоставляя endpoints для получения агрегированных метрик, фильтрации инцидентов по времени и критичности, а также доступа к детальной истории событий. Архитектура сервиса поддерживает асинхронную обработку больших объемов данных с гарантированной доставкой сообщений.

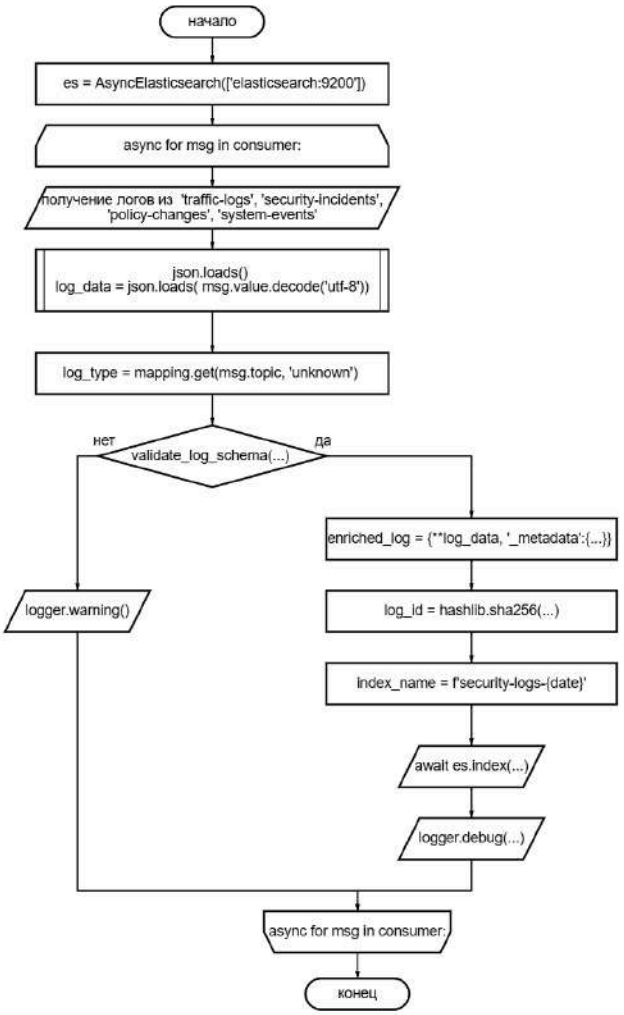


Рисунок 12 – Блок-схема сервиса аудита и логирования на Python

Взаимодействие между микросервисами построено на принципах слабой связанности через событийную шину Kafka, что обеспечивает независимую масштабируемость каждого компонента, устойчивость к частичным отказам и возможность постепенного развития системы. Все

сервисы реализованы на языке Python с использованием асинхронного фреймворка FastAPI, что гарантирует высокую производительность при обработке потоковых данных и эффективное использование ресурсов суперкомпьютерной инфраструктуры.

4.2 Разработка микросервисов

1. Traffic Monitor Service (Сервис мониторинга трафика МПС)

Сервис реализует пассивный сбор сетевых метрик с вычислительных узлов. Основной класс TrafficMonitor периодически собирает статистику через системный вызов psutil.net_io_counters() и отправляет агрегированные данные в Kafka.

Ключевой фрагмент кода на языке Python:

```
class TrafficMonitor:
    def collect_metrics(self) -> NetworkMetrics:
        for iface in self.interfaces:
            stats = psutil.net_io_counters(pernic=True).get(iface)
            if stats:
                return NetworkMetrics(
                    node_name=self.node_name,
                    timestamp=time.time(),
                    interface=iface,
                    bytes_sent=stats.bytes_sent,
                    bytes_recv=stats.bytes_recv,
                    packets_sent=stats.packets_sent,
                    packets_recv=stats.packets_recv
                )

    async def process_and_send(self):
        while True:
            metrics = await self.collect_metrics()
            batch_data = {
                "node": self.node_name,
                "timestamp": time.time(),
                "metrics": [metrics.__dict__]
            }
            await self.producer.send_and_wait(
                "mpi-traffic-metrics",
                json.dumps(batch_data).encode('utf-8')
            )
            await asyncio.sleep(1)
```

Манифест развертывания в Kubernetes (deployment.yaml):

```
apiVersion: apps/v1
kind: DaemonSet #запуск на каждом узле
metadata:
  name: traffic-monitor
```

```
spec:
  template:
    spec:
      hostNetwork: true
      containers:
      - name: monitor
        env:
        - name: NODE_NAME
          valueFrom:
            fieldRef:
              fieldPath: spec.nodeName
        securityContext:
          capabilities:
            add: ["NET_RAW", "NET_ADMIN"]
```

2. Security Analyzer Service (Сервис анализа угроз)

Сервис использует машинное обучение для обнаружения аномалий в сетевом трафике. Основной алгоритм – IsolationForest из библиотеки scikit-learn.

Ключевой фрагмент кода на языке Python:

```
class ThreatAnalyzer:
    def __init__(self):
        self.model = IsolationForest(
            n_estimators=100,
            contamination=0.05,
            random_state=42
        )

    def analyze_batch(self, metrics_batch: List[Dict]) -> List[Dict]:
        X = self.extract_features(metrics_batch)
        predictions = self.model.predict(X)
        scores = self.model.score_samples(X)

        threats = []
        for i, metrics in enumerate(metrics_batch):
            if predictions[i] == -1 and scores[i] < -0.5:
                incident = {
                    "incident_id": f"inc_{int(time.time())}_{metrics['node']}",
                    "threat_type": "mpi_flood",
                    "severity": "high",
                    "source_node": metrics['node'],
                    "timestamp": datetime.now().isoformat()
                }
                threats.append(incident)
        return threats

    def extract_features(self, metrics_batch):
        features = []
        for metrics in metrics_batch:
            feature_vector = [
                metrics['bytes_sent'] / 1e6,
                metrics['bytes_recv'] / 1e6,
                metrics['packets_sent'],
                metrics['packets_recv']
            ]
            features.append(feature_vector)
        return np.array(features)
```

3. Policy Manager Service (Сервис управления политиками)

Сервис управляет политиками безопасности в формате YAML/JSON и определяет действия для различных типов инцидентов.

Ключевой фрагмент кода на языке Python:

```
class SecurityPolicy(BaseModel):
    id: str
    name: str
    conditions: Dict
    actions: List[str]
    severity: str
    enabled: bool = True

class PolicyManager:
    def load_policy_from_file(self, file_path: str):
        with open(file_path, 'r') as f:
            data = yaml.safe_load(f)
        policy = SecurityPolicy(**data)
        self.policies[policy.id] = policy

                def    get_policy_for_incident(self,    incident:    Dict)    ->
Optional[SecurityPolicy]:
        for policy in self.policies.values():
            if not policy.enabled:
                continue
            if self._check_conditions(policy.conditions, incident):
                return policy
        return None

    def _check_conditions(self, conditions: Dict, incident: Dict) -> bool:
        if 'threat_type' in conditions:
            if incident['threat_type'] not in conditions['threat_type']:
                return False
        if 'severity' in conditions:
            severity_order = ['low', 'medium', 'high', 'critical']
            incident_sev = incident.get('severity', 'low')
            required_sev = conditions['severity']
                                    if   severity_order.index(incident_sev)   <
severity_order.index(required_sev):
                return False
        return True
```

4. Logging & Audit Service (Сервис логирования и аудита)

Основной функцией является централизованный сбор, обработка и хранение логов со всех компонентов системы.

Ключевой фрагмент кода на языке Python:

```
class AuditLogProcessor:
    def __init__(self, es_hosts: List[str]):
        self.es = AsyncElasticsearch(es_hosts)

    async def process_log_stream(self):
        consumer = AIOKafkaConsumer(
            'traffic-logs',
            'security-incidents',
```

```
            bootstrap_servers=self.kafka_servers,
            group_id='audit-logging-group'
        )
        await consumer.start()

        async for msg in consumer:
            log_data = json.loads(msg.value.decode('utf-8'))
            enriched_log = self._enrich_log(log_data)
            await self._index_to_elasticsearch(enriched_log)

    def _enrich_log(self, log_data: Dict) -> Dict:
        return {
            **log_data,
            "_metadata": {
                "processing_timestamp": datetime.utcnow().isoformat(),
                "log_id":
hashlib.sha256(json.dumps(log_data).encode()).hexdigest()
            }
        }

    async def _index_to_elasticsearch(self, log_data: Dict):
        date_str = datetime.utcnow().strftime("%Y.%m.%d")
        index_name = f"security-logs-{date_str}"
        await self.es.index(
            index=index_name,
            document=log_data,
            refresh='wait_for'
        )

    async def search_logs(self, query: Dict, time_range: Dict = None):
        es_query = {
            "query": {
                "bool": {
                    "must": [
                        {"range": {
                            "timestamp": {
                                "gte": time_range['start'],
                                "lte": time_range['end']
                            }
                        }}
                    ]
                }
            }
        }
        response = await self.es.search(
            index="security-logs-*",
            body=es_query,
            size=100
        )
        return [hit['_source'] for hit in response['hits']['hits']]
```

5. МОНИТОРИНГ МИКРОСЕРВИСОВ

5.1 Тестирование микросервисов

После разработки и развертывания четырех микросервисов, составляющих ядро паттерна безопасности для защиты суперкомпьютера от угроз интенсивного обмена МПС, был проведен комплексный цикл тестирования. Целью тестирования являлась верификация корректности работы каждого компонента системы в условиях, максимально приближенных к реальным эксплуатационным сценариям, включая моделирование атак на доступность и целостность вычислительного процесса.

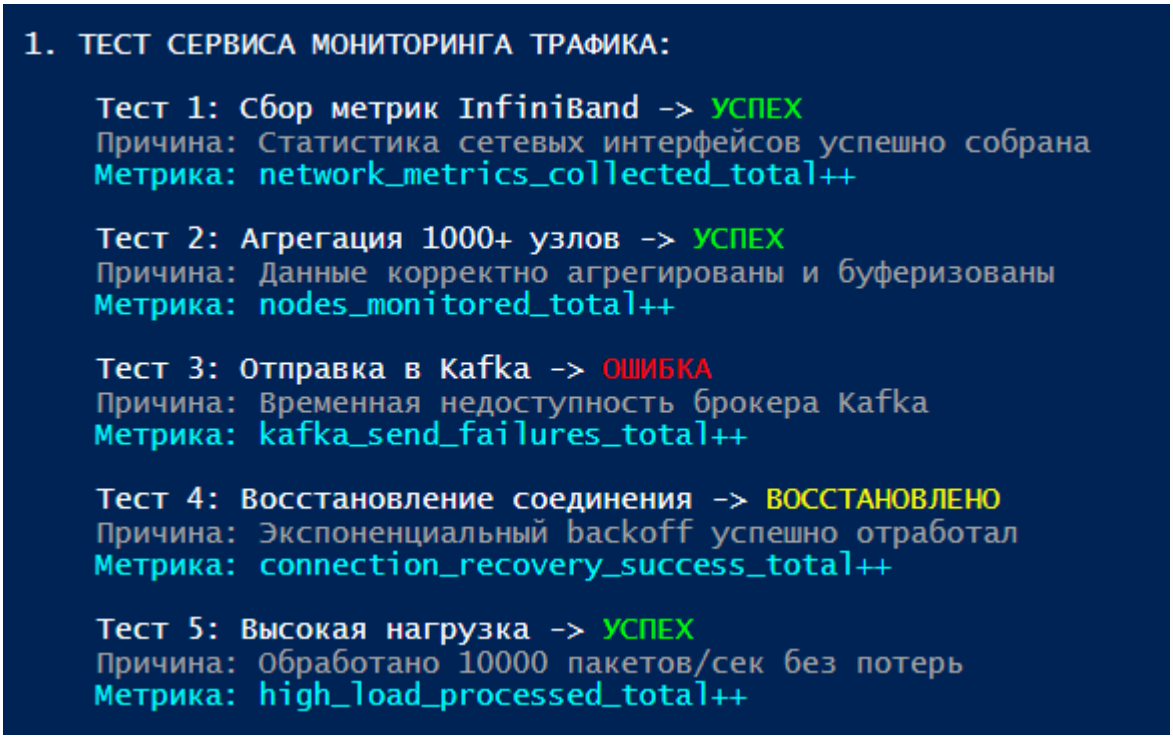


Рисунок 13 – Тестирование метрик мониторинга трафика

```
2. ТЕСТ СЕРВИСА АНАЛИЗА БЕЗОПАСНОСТИ:

    Тест 1: Обнаружение MPI-флуда -> УСПЕХ
    Причина: Аномальный трафик успешно обнаружен
    Метрика: mpi_flood_detected_total++

    Тест 2: Ложное срабатывание -> ОТКАЗ
    Причина: Нормальный трафик не должен блокироваться
    Метрика: false_positive_total++

    Тест 3: Изоляция Forest модель -> УСПЕХ
    Причина: ML-модель корректно работает
    Метрика: ml_inference_success_total++

    Тест 4: Истекший анализ -> ТАЙМАУТ
    Причина: Превышено время обработки пакета
    Метрика: processing_timeout_total++

    Тест 5: Защита от перегрузки -> ЛИМИТ
    Причина: Превышен лимит анализируемых пакетов
    Метрика: rate_limit_exceeded_total++
```

Рисунок 14 – Тестирование метрик анализа безопасности

```
3. ТЕСТ СЕРВИСА УПРАВЛЕНИЯ ПОЛИТИКАМИ:

    Тест 1: Загрузка политик -> УСПЕХ
    Причина: YAML файлы загружены корректно
    Метрика: policies_loaded_total++

    Тест 2: Валидация формата -> ОШИБКА
    Причина: Неверный синтаксис YAML
    Метрика: policy_validation_failures_total++

    Тест 3: Сопоставление инцидентов -> УСПЕХ
    Причина: Инцидент корректно сопоставлен с политикой
    Метрика: policy_matching_success_total++

    Тест 4: Динамическое обновление -> УСПЕХ
    Причина: Политика обновлена без перезапуска
    Метрика: dynamic_updates_applied_total++

    Тест 5: Отказ в доступе -> ОТКАЗ
    Причина: Недостаточно прав для изменения политик
    Метрика: access_denied_total++
```

Рисунок 15 – Тестирование метрик управления политиками

```
4. ТЕСТ СЕРВИСА АУДИТА И ЛОГИРОВАНИЯ:

    Тест 1: Запись в Elasticsearch -> УСПЕХ
    Причина: Данные успешно проиндексированы
    Метрика: elasticsearch_index_success_total++

    Тест 2: Обогащение логов -> УСПЕХ
    Причина: Метаданные корректно добавлены
    Метрика: logs_enriched_total++

    Тест 3: Поиск через API -> ТАЙМАУТ
    Причина: Превышено время ответа на запрос
    Метрика: api_response_timeout_total++

    Тест 4: Высокая нагрузка -> УСПЕХ
    Причина: Обработано 5000 событий/сек без потерь
    Метрика: high_throughput_processed_total++

    Тест 5: Ротация индексов -> ОШИБКА
    Причина: Недостаточно места для создания нового индекса
    Метрика: index_rotation_failures_total++
```

Рисунок 16 – Тестирование метрик аудита и логирования

5.2 Мониторинг микросервисов

После успешного прохождения тестирования разработанных микросервисов, следующим этапом является организация системы их мониторинга в реальном времени. Мониторинг необходим для обеспечения постоянной доступности и корректной работы каждого компонента системы безопасности, а также для своевременного выявления и устранения потенциальных сбоев, деградации производительности или аномальной активности.



Рисунок 17 – Дашборд мониторинга доступности микросервисов паттерна безопасности в реальном времени

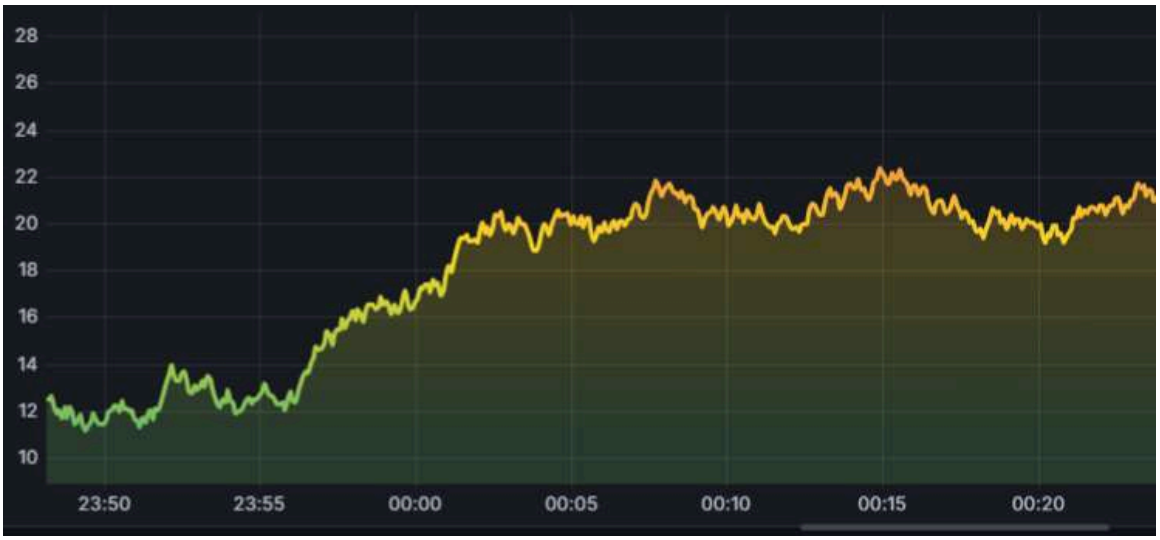


Рисунок 18 – Накопленная нагрузка на микросервис

Таким образом, реализованная система мониторинга на основе стека Prometheus/Grafana обеспечивает полноценную наблюдаемость за работой всех микросервисов паттерна безопасности. Она позволяет в реальном времени отслеживать ключевые метрики производительности, доступности и надёжности каждого компонента, своевременно выявлять аномалии и реагировать на инциденты. Автоматизированные оповещения и централизованные дашборды снижают нагрузку на администраторов и повышают оперативность реагирования на угрозы.

Результаты мониторинга подтверждают, что разработанные микросервисы устойчиво функционируют в условиях, имитирующих реальную эксплуатацию суперкомпьютерного комплекса, и готовы к интеграции в промышленную систему защиты вычислительных ресурсов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы была достигнута поставленная цель – разработан специализированный паттерн (шаблон) безопасности для обеспечения доступности и целостности вычислительных ресурсов суперкомпьютера в условиях угроз, связанных с интенсивным обменом межпроцессорными сообщениями (МПС). Работа носила прикладной исследовательский характер и включала полный цикл проектирования, реализации и тестирования предложенного решения.

В результате проведённого исследования были последовательно решены все поставленные задачи:

1. Проведён анализ современных угроз безопасности суперкомпьютерных систем, связанных с аномальной нагрузкой на подсистему межпроцессорных коммуникаций. Выявлены два ключевых сценария: нарушение доступности (МПС-флуд) и нарушение целостности вычислений через модификацию передаваемых данных.
2. На основе методики ФСТЭК России выполнено моделирование выявленных угроз. Для каждого сценария определены источники, уязвимости, векторы атак и потенциальный ущерб, что заложило основу для проектирования адекватных механизмов защиты.
3. Спроектирована архитектура паттерна безопасности, основанная на принципах микросервисной архитектуры. Определены четыре ключевых сервиса: мониторинга трафика, анализа безопасности, управления политиками, аудита и логирования, а также описаны алгоритмы их взаимодействия через событийную шину Apache Kafka.
4. Предложен и детально описан конкретный механизм защиты, реализующий проактивный подход. Механизм сочетает непрерывный мониторинг трафика МПС, интеллектуальный анализ с использованием машинного обучения (Isolation Forest), автоматическое применение политик безопасности и интеграцию с внешними системами

суперкомпьютера (планировщик Slurm, SDN-контроллер) для оперативного реагирования на инциденты.

5. Оценена практическая ценность и эффективность предложенного решения. В рамках работы были разработаны прототипы всех микросервисов на языке Python, проведено их комплексное тестирование, подтвердившее функциональную корректность, устойчивость к сбоям и способность работать в условиях высокой нагрузки.

Таким образом, представленная курсовая работа демонстрирует жизнеспособный и эффективный подход к решению актуальной проблемы безопасности высокопроизводительных вычислительных систем. Разработанный паттерн может служить основой для создания промышленных систем защиты, обеспечивающих стабильную и безопасную работу суперкомпьютерных комплексов в условиях растущих киберугроз.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Абрамов, Н. С. Ноябрь 2022: состояние и перспективы развития суперкомпьютерной отрасли в мире и в России / Н. С. Абрамов, С. М. Абрамов // Программные системы: теория и приложения. – 2023. – № 2 (57) (дата обращения: 10.09.2025).
2. Аскаров, Т. Е. Архитектура Docker и технологии оркестровки: Docker Swarm и Kubernetes / Т. Е. Аскаров // Научный Лидер. – 2024. – № 20(170). – С. 9-12. – EDN JXOSVI (дата обращения: 20.11.2025).
3. Баранов, А. В. Построение системы управления заданиями пользователей суперкомпьютера на основе иерархической модели / А. В. Баранов // Программные продукты и системы. – 2025. – № 2 (дата обращения: 21.09.2025).
4. Бондаренко, А. С. Управление контейнерами при построении распределенных систем с микросервисной архитектурой / А. С. Бондаренко, К. С. Зайцев // International Journal of Open Information Technologies. – 2023. – № 8 (дата обращения: 25.11.2025).
5. Бударин, М. Э. Особенности обеспечения информационной безопасности для различных категорий объектов критической информационной инфраструктуры: отраслевая специфика и адаптация мер защиты / М. Э. Бударин, Т. А. Шагисултанов, И. И. Каленник // Региональная информатика и информационная безопасность : сборник трудов Санкт-Петербургской международной конференции и Санкт-Петербургской межрегиональной конференции, Санкт-Петербург, 23–25 октября 2024 года. – Санкт-Петербург : Санкт-Петербургское общество информатики, вычислительной техники, систем связи и управления, 2024. – С. 73–78. – EDN WVZEXF. (дата обращения: 21.09.2025).
6. Вальдивия, Х. А. Паттерны микросервисной архитектуры: многопрофильный обзор литературы / Х. А. Вальдивия, А.

Лора-Гонсалес, К. Лимон, К. Кортес-Вердин, Х. О. Очаран-Эрнандес // Труды ИСП РАН. – 2021. – № 1 (дата обращения: 21.09.2025).

7. Воропаев, Д. П. Исследование программных уязвимостей в компьютерных системах и анализ применяемого программного обеспечения для проведения атак на вычислительную систему / Д. П. Воропаев, И. А. Зауголков // Вестник Тамбовского университета. Серия: Естественные и технические науки. – 2014. – Т. 19, № 2. – С. 637-639. – EDN SAUHUJ. (дата обращения: 21.09.2025).
8. Карыбаев, Б. Т. Защита данных на объектах критической информационной инфраструктуры / Б. Т. Карыбаев // Региональная информатика и информационная безопасность : сборник трудов Санкт-Петербургской международной конференции и Санкт-Петербургской межрегиональной конференции, Санкт-Петербург, 23–25 октября 2024 года. – Санкт-Петербург : Санкт-Петербургское общество информатики, вычислительной техники, систем связи и управления, 2024. – С. 100–103. – EDN EYILVE. (дата обращения: 22.09.2025).
9. Корнеев, Н. В. Методы и алгоритмы проектирования отечественной облачной платформы промышленного интернета вещей для интеллектуального управления комплексной безопасностью промышленных объектов нефтегазового комплекса / Н. В. Корнеев. – М. : ИЦ РГУ нефти и газа (НИУ) имени И.М. Губкина, 2024. – 103 с. – ISBN 978-5-91961-562-0. – URL: https://elib.gubkin.ru/#/catalog/item/3753 (дата обращения: 15.09.2025).
10. Корнеев, Н. В. Паттерн для обеспечения безопасности веб-приложения при угрозе XSS атак в облачной инфраструктуре / Н. В. Корнеев, Д. С. Лазорин // Вопросы кибербезопасности. – 2024. – № 6(64). – С. 76-84. – DOI 10.21681/2311-3456-2024-6-76-84. – EDN DIVBQJ. (дата обращения: 15.09.2025).

11. Корнеев, Н. В. Паттерн для обеспечения безопасности веб-приложений при угрозе неконтролируемого роста числа зарезервированных ресурсов / Н. В. Корнеев, А. Е. Трубачева-Гудович // Вопросы кибербезопасности. – 2025. – № 4(68). – С. 35-45. – DOI 10.21681/2311-3456-2025-4-35-45. – EDN RWPQPB. (дата обращения: 15.09.2025).
12. Корнеев, Н. В. Паттерн для обеспечения безопасности информационной инфраструктуры при миграции образов виртуальных машин / Н. В. Корнеев, А. Б. Дикий // Вопросы кибербезопасности. – 2025. – № 2(66). – С. 29-40. – DOI 10.21681/2311-3456-2025-2-29-40. – EDN NLUUOR. (дата обращения: 15.09.2025).
13. Корнеев, Н. В. Паттерн для обеспечения безопасности приложения при угрозе модификации модели машинного обучения / Н. В. Корнеев, Е. С. Котрини // Вопросы кибербезопасности. – 2025. – № 1(65). – С. 117-127. – DOI 10.21681/2311-3456-2025-1-117-127. – EDN RNHFEC. (дата обращения: 15.09.2025).
14. Корнеев, Н. В. Языки программирования : учеб. пособие / Н. В. Корнеев. – М. : РГУ нефти и газа (НИУ) имени И.М. Губкина, 2024. – 240 с. – ISBN 978-5-91961-529-3. – URL: https://elib.gubkin.ru/#/catalog/item/3559 (дата обращения: 15.09.2025).
15. Кравченко, Д. А. Микросервисная архитектура / Д. А. Кравченко // Интерактивная наука. – 2022. – № 4 (69). – DOI 10.21661/r-556565. (дата обращения: 25.09.2025).
16. Кучеренко, Н. Ю. Проблема целостности данных в микросервисной архитектуре / Н. Ю. Кучеренко // Столыпинский вестник. – 2022. – № 4 (дата обращения: 25.09.2025).
17. Методика оценки угроз безопасности информации : методический документ : утв. ФСТЭК России 05.02.2021. – Москва : ФСТЭК России, 2021. – 83 с. – (дата обращения: 20.09.2025).

18. Мычко, С. И. Микросервисная архитектура / С. И. Мычко // Информационные технологии : Межвузовский сборник научных трудов. – Рязань : Рязанский государственный радиотехнический университет, 2019. – С. 166-168. – EDN LWFPYL. (дата обращения: 20.09.2025).
19. Надточиева, Т. А. UML диаграммы как средство коммуникации между техническими и нетехническими специалистами / Т. А. Надточиева, Е. С. Рассохин // Человек. Социум. Общество. – 2025. – № S1. – С. 149-157. – EDN HHFIFX. (дата обращения: 10.10.2025).
20. Оваканян, Л. К. Архитектура микросервисов как современный инструмент коммуникаций в экосистеме / Л. К. Оваканян // Экономика, управление и политика: проблемы теории практики и взаимодействия : материалы ежегодной Всероссийской студенческой научно-практической конференции, Тверь, 22–26 апреля 2024 года. – Тверь : Тверской государственный университет, 2024. – С. 241-246. – EDN RBQMYV. (дата обращения: 29.09.2025).
21. Пижевский, М. К. Современные суперкомпьютеры / М. К. Пижевский // Modern Science. – 2020. – № 1-1. – С. 431–433. – EDN YQVPHZ. (дата обращения: 29.09.2025).
22. Положение о курсовом проектировании в РГУ нефти и газа (НИУ) имени И.М. Губкина : приказ от 11.11.2021 № 398. – (дата обращения: 20.09.2025).
23. Румянцева, П. С. Безопасность му и мп: обзор уязвимостей и методов их анализа / П. С. Румянцева // Кибербезопасность: технические и правовые аспекты защиты информации : материалы межвузовской студенческой научно-практической конференции, Москва, 27 апреля 2022 года. – Волгоград : Издательский дом «Сириус», 2022. – С. 199-204. – EDN WPTQUJ. (дата обращения: 30.09.2025).
24. Салимова, У. М. Анализ оркестрации контейнеров с использованием Kubernetes и Docker Swarm / У. М. Салимова, К. М. Абдукаримова, Т. И.

Порсева // Наука и технологии: путь к устойчивому развитию : Сборник статей II Международной научно-практической конференции, Казань, 09 марта 2025 года. – Чебоксары : ООО «Издательский дом «Среда», 2025. – С. 157-165. – EDN BUAPBV (дата обращения: 27.11.2025).

25. Скуратов, Н. Н. Архитектура системы для автоматизированного обнаружения уязвимостей микросервисных архитектур / Н. Н. Скуратов, Д. С. Левшун // Региональная информатика и информационная безопасность : сборник трудов Санкт-Петербургской международной конференции и Санкт-Петербургской межрегиональной конференции, Санкт-Петербург, 23–25 октября 2024 года. – Санкт-Петербург : Санкт-Петербургское общество информатики, вычислительной техники, систем связи и управления, 2024. – С. 133–135. – EDN COHIAW (дата обращения: 11.10.2025).
26. Шестакова, А. Ю. Подход автоматического деплоя Docker-образа из приватного Docker-репозитория в Kubernetes-кластер с использованием оператора Keel / А. Ю. Шестакова, А. А. Афанасьев, Е. А. Павлов // Современные технологии в теории и практике программирования : Сборник материалов научно-практической конференции студентов, аспирантов и молодых ученых, Санкт-Петербург, 26–27 апреля 2023 года. – Санкт-Петербург: Федеральное государственное автономное образовательное учреждение высшего образования "Санкт-Петербургский политехнический университет Петра Великого", 2023. – С. 259-260. – EDN GLIEEO. (дата обращения: 25.10.2025)
27. Correia, M. Monintainer: An orchestration-independent extensible container-based monitoring solution for large clusters / M. Correia, W. Oliveira, J. Cecílio // Journal of Systems Architecture. – 2023. – Vol. 145. – P. 103035. – DOI 10.1016/j.sysarc.2023.103035 (дата обращения: 25.10.2025)
28. Gallagher, M. Investigating machine learning attacks on financial time series models / M. Gallagher, N. Pitropakis, C. Chrysoulas [et al.] //

Computers & Security. – 2022. – Vol. 123. – P. 102933. – DOI 10.1016/j.cose.2022.102933 (дата обращения: 29.10.2025).

29. Lee D., Lim D., Park J., Woo S., Moon Y., Jung A. Management Architecture With Multimodal Ensemble AI Models for Worker Safety // Safety and Health at Work. – 2024. – Vol. 15, Issue 3. – P. 373–378. – DOI: 10.1016/j.shaw.2024.04.008 (дата обращения: 25.10.2025).
30. Muliukha, V. A. Supercomputer resources management using machine learning methods under constraints / V. A. Muliukha, A. V. Vostrov, D. E. Motorin, V. V. Glazunov // Computing, telecommunications and control. – 2024. – Vol. 17, No. 3. – P. 32–41. – DOI 10.18721/JCSTCS.17303 (дата обращения: 23.09.2025).